

SOFTWARE TEST PLAN

V.A.R.I.O.

Voice Adaptive Role & Intent Orchestrator

Team 05 PIPELINE (Project ID 42)

Rohith G S · Kurumeti Mitesh · Mohammed Tahir Siddique · Pavan Kishor M

UE24CS341A — Software Engineering, PES University

CONTENTS

1.	Introduction	3
1.1	Project and Test Strategy Overview	3
1.2	Test Entry Criteria	3
1.3	Test Exit Criteria / Test Completeness	3–4
1.4	Scope	5
1.5	Quality Objectives	6
1.6	Team Roles and Testing Responsibilities	6
2.	Test Methodology	7
2.1	Working Method	7
2.2	Test Levels	7
2.3	Test Case ID Convention	7
2.4	Test Case Design Rules	7–8
2.5	Defect Triage	9
2.6	Suspension Criteria and Resumption	9
2.7	Test Completeness	9
2.8	Non-Functional Test Method	10
3.	Test Deliverables	11
3.1	Test Result Status Values	11
4.	Resource & Environment Needs	12
4.1	Testing Tools	12
4.2	Test Environment Model	12
4.3	Deployment/Test Safeguards	12
4.4	Test Data Management	12–13
Appendix A	Functional Requirement Traceability	14
Appendix B	Sprint Exit-Criteria Test Checklist	15
Appendix C	Test Risk Register	16
Appendix D	Team Review and Sign-off	17

1. Introduction

1.1 Project and Test Strategy Overview

V.A.R.I.O. (Voice Adaptive Role & Intent Orchestrator) is a reusable, role-specific conversational assistant framework supporting HR, IT Support, and Admissions role packs on a shared core. The shared core provides session management, dialogue routing, response rendering, and common web/application services, while each role pack supplies department-specific vocabulary and workflows.

The test strategy follows the same architecture: validate small units first, then shared components and contracts, then role-pack integrations, then complete end-to-end workflows, followed by non-functional, regression, and final acceptance checks. Testing is continuous across all four sprints rather than deferred to the end of the project.

Stage	Primary purpose	Evidence
Unit	Verify isolated backend/frontend logic	pytest / Jest results
Component / UI	Verify shared message components and admin views	Component/UI checks
Integration	Verify adapters, Rasa, gateway, database and mocks work together	Integration test results
API / Contract	Verify envelopes, validation, authentication, status codes and rate limits	API test collection/results
System / E2E	Verify complete user journeys across the deployed application	End-to-end test cases
Non-functional	Verify performance, security, reliability and degradation	Performance/security/reliability records
Regression	Confirm previous acceptance criteria remain valid	CI + sprint regression sweep
User acceptance	Whole-team FR-by-FR demonstration and rehearsal	UAT checklist + team sign-off

1.2 Test Entry Criteria

Testing for a slice or sprint begins only when the minimum conditions below are satisfied. These criteria prevent incomplete builds or unavailable dependencies from being mistaken for test failures.

ID	Entry criterion
ENT-01	Scope and acceptance criteria for the slice are defined and understood by the owner.
ENT-02	The code builds and the relevant application services can start in the intended test environment.
ENT-03	Required mock services, seeded test data, configuration and non-secret environment variables are available.
ENT-04	The automated CI baseline is available for the branch/PR; known unrelated failures are documented before execution.
ENT-05	For integration/system tests, dependent role-pack, gateway, database and mock-service paths required by the case are reachable.
ENT-06	For security-sensitive tests, the test accounts and role scopes required to verify authorization are available.
ENT-07	For performance tests, the deployed test target and controlled workload/data set are available.

1.3 Test Exit Criteria / Test Completeness

A slice is test-complete only when its acceptance criteria have evidence, applicable automated tests are green, defects are triaged, and peer review is complete. A sprint is test-complete when its exit criteria and regression checks pass. The project is test-complete when all requirements and sprint gates below are satisfied.

ID	Exit criterion
EXT-01	Every in-scope slice acceptance criterion has a linked test case and recorded result.
EXT-02	Every FR-1 through FR-19 has at least one passing automated or documented manual test.
EXT-03	No open Critical or High defects remain at the sprint/project exit; Medium/Low items are explicitly triaged and deferred only when they do not violate an acceptance criterion.
EXT-04	Main-branch CI is green and the required branch/deployment gates are functioning.
EXT-05	Security, authorization, confirmation, validation and audit-log checks pass for applicable workflows.

ID	Test criterion
EXT-06	Performance target is met: p95 response time < 2 seconds excluding artificial mock-API delay, with 20 concurrent sessions showing no noticeable degradation.
EXT-07	Graceful degradation and recovery behavior has been exercised for applicable mock-service, TTS and transient backend failures.
EXT-08	The final 20-session demonstration/rehearsal is completed and the four team members sign the final test closure.

1.4 Scope

In Scope

Area	In-scope test targets	Priority index
Authentication	Delegated login through mock authentication service and adapter; VARIO JWT issuance; user upsert-on-login; login loading/error states	01
Database / schema	PostgreSQL schema, migrations, foreign keys, JSONB, enum/check constraints and integrity	02
API gateway	Response envelope, JWT middleware, validation, error mapping, rate limiting and health endpoint	03
Conversation store	Conversation creation/listing, owner-scoped history, pagination and UI flows	04
Chat core	Session/slot persistence, dialogue routing, response rendering, confidence clarification, classification/routing logs and shared message components	05
Talk mode	TTS, browser STT, microphone states, hands-free loop and TTS-unavailable fallback	06
Admin console	Role-based authorization, role-pack scoping, navigation, intent management and audit-log viewing	07–08
Rephraser	Variant generation, placeholder preservation/validation, training-phrase generation and graceful LLM failure	09
Retraining	Retrain trigger/status, Rasa YAML build/train/reload and needs_retrain handling	10
Conversation logs	Role-scoped transcript viewing, intent/confidence display and pagination	11
HR role pack	NLU, HRMS adapter/mock, leave balance, leave workflow, confirmation/audit and policy FAQ	12–16
IT role pack	NLU, ITSM adapter/mock, ticket status, ticket creation/deduplication and password-reset workflow	17–21
Admissions role pack	NLU, admissions adapter/mock, application status, checklist and fee/deadline answers	22–26
Infrastructure / CI/CD	Local Compose stack, automated CI, branch protection, OCI/Vercel deployment path, Cloudflare HTTPS and Neon environments	27–30
Cross-cutting NFRs	20-session performance test, input validation/sanitization, graceful degradation and session recovery	31

Out of Scope for v1

Excluded item	Boundary
Real enterprise integrations	Real HRMS, ITSM, Admissions or identity-provider integrations; v1 uses mock services behind adapters.
Native authentication features	User registration, email verification, native password reset and account lockout.
Multilingual support	English-only v1.
Native mobile	Responsive web only; desktop and mobile breakpoints are tested.
Advanced analytics / BI	Only basic audit and conversation-log views are included.
Proactive notifications	The system responds to user-initiated queries only.
Server-side STT / voice selection	Browser STT and the configured default Neural2 voice are used.
Scheduled retraining	Retraining is manually triggered.
SonarQube gate	Optional future CI capability; not part of v1 acceptance.
Kubernetes / autoscaling / blue-green / production-scale load	The target is the 20-session demonstration scale.
Payments	No payment feature exists in the product scope.
Real credential/password handling	Password reset is a mocked workflow and does not manage real credentials.

1.5 Quality Objectives

ID	Objective	Target / acceptance	Verification
QO-1	Performance	p95 response time < 2 seconds, excluding artificial mock-API delay	Performance test
QO-2	Scalability	20 concurrent sessions with no noticeable degradation	Concurrent-session test
QO-3	Extensibility	A new role pack can be introduced through configuration/module boundaries without core-engine changes; design-time verification is used for v1	Architecture/module verification
QO-4	Functional completeness	FR-1 through FR-19 complete successfully through the conversational/application interface	System/E2E + FR traceability
QO-5	Safety	Record-affecting workflows require explicit confirmation and are audit logged	Negative + confirmation + audit tests
QO-6	Security	JWT-protected routes, role-pack scoping, input validation/sanitization and no cross-role data access	API/security tests
QO-7	Reliability	Graceful degradation for mock/TTS failures and session recovery after transient backend restart	Fault-injection/recovery tests
QO-8	Coverage	Every FR has at least one automated or documented manual test	RTM review

1.6 Team Roles and Testing Responsibilities

Testing is a shared engineering responsibility across the four-person team. There is no separate QA team or external testing role. The member who owns a slice is accountable for creating and executing its initial tests; a second team member independently reviews the evidence and relevant acceptance criteria. The team jointly performs regression, integration, system, performance and final demonstration testing.

Member	Primary engineering ownership	Testing responsibility
Rohith G S	Platform/core and shared application areas	Tests core behavior, integration boundaries, authentication/security-sensitive flows as applicable; also performs peer reviews for role-pack and infrastructure changes.
Kurumeti Mitesh	HR role pack	Owns HR functional tests, NLU/adaptor checks, confirmation/audit checks and HR regression; peer-reviews non-HR work.
Mohammed Tahir Siddique	IT role pack and related delivery areas	Owns IT functional tests, ticket/password-reset safety checks and IT regression; peer-reviews other slices.
Pavan Kishor M	Admissions role pack and infrastructure/CI-CD areas	Owns Admissions tests and deployment/CI validation; peer-reviews other slices.
All four	Shared project responsibility	Run cross-cutting regression, security/authorization checks, performance/reliability tests, final FR walkthrough, defect triage and final test closure/sign-off.

Peer-review rule: the implementer does not provide the only approval for their own slice. At least one other team member verifies the acceptance evidence before the slice is marked complete.

2. Test Methodology

2.1 Working Method

VARIO uses Agile, slice-based development with continuous delivery. Testing begins from acceptance criteria, uses contract-first checks for shared APIs, isolates role-pack behavior before end-to-end wiring, and continuously exercises the main integration path. Test evidence is kept with the relevant slice, sprint, CI run, defect record, or test report.

2.2 Test Levels

Level	Scope	Typical methods	Timing / ownership
1. Unit	Pure backend/frontend logic and utilities	pytest, Jest	During implementation; slice owner
2. Component / UI	Shared message components, admin views and UI states	React Testing Library, Storybook-style component checks	During UI work; owner + peer
3. Integration	Adapters ↔ mocks, router ↔ Rasa, renderer ↔ DB and related boundaries	pytest/httpx, service integration tests, fault injection	As dependencies become available
4. API / Contract	Envelope, payload shape, auth, validation, status codes and rate limits	pytest/httpx, curl/Postman-equivalent collection	From gateway availability; continuous regression
5. System / E2E	Complete login, chat, role-pack, admin, write and voice journeys	Browser/manual E2E and scripted workflows	Every sprint boundary; full sweep in Sprint 4
6. Non-functional	Performance, security, reliability and degradation	Concurrent-session harness, negative tests, restart/fault injection	Early enough to leave repair time
7. Regression	Previously passing behavior after change	Full CI + targeted acceptance sweep	Every PR and sprint boundary
8. UAT / acceptance	Whole-team FR-by-FR walkthrough of deployed application	Scripted 20-session demonstration/rehearsal	Final Sprint 4 window

2.3 Test Case ID Convention

Every planned test case receives a stable identifier so that requirements can be traced to executable evidence and results. IDs identify the test domain, not the implementation file.

Pattern	Meaning	Example
TC-AUTH-###	Authentication	TC-AUTH-001
TC-CORE-###	Shared chat/session/routing	TC-CORE-001
TC-HR-###	HR role pack	TC-HR-001
TC-IT-###	IT role pack	TC-IT-001
TC-ADM-###	Admissions role pack	TC-ADM-001
TC-ADMIN-###	Admin console/configuration	TC-ADMIN-001
TC-API-###	API/contract	TC-API-001
TC-SEC-###	Security/authorization/input safety	TC-SEC-001
TC-PERF-###	Performance/load	TC-PERF-001
TC-REL-###	Reliability/degradation/recovery	TC-REL-001
TC-CICD-###	CI/CD/deployment	TC-CICD-001
TC-E2E-###	Cross-role/system workflow	TC-E2E-001

2.4 Test Case Design Rules

- Each acceptance criterion gets at least one positive test and, where applicable, a negative or boundary test.
- Record-affecting workflows include explicit confirmation, cancellation/negative-path checks and audit-log verification.

- Authorization tests verify both allowed access and denied cross-role access.
- API tests verify success and relevant 400/401/403/404/409/422/429 behavior where the endpoint supports those outcomes.
- Failure-path tests cover unavailable mocks, unavailable TTS, malformed input, missing records and transient backend restart where applicable.
- UI tests cover loading, error, empty and success states for user-visible flows.

2.5 Defect Triage

Every defect is recorded with the affected requirement/slice, test case ID, severity, priority, evidence, status and resolution notes. Severity measures impact; priority determines when the team must act relative to sprint commitments.

Severity	Description	Valid examples	Response
Critical / Sev-1	System-down, security/authorization breach, data loss/corruption or unsafe record-affecting behavior	JWT bypass; cross-role data access; unconfirmed leave/ticket/password-reset action; migration corruption; real credential leakage	Stop affected work; fix before merge/deploy; peer verification required
High / Sev-2	Major functional failure, crash/hang, broken core workflow or session persistence	Core chat unavailable; role-pack flow cannot complete; repeated deployment failure blocking sprint exit	Fix before affected slice/sprint exit
Medium / Sev-3	Partial or non-blocking behavior issue	Limited UI state or recoverable edge case	Fix or explicitly defer with team decision
Low / Sev-4	Cosmetic/minor issue without functional impact	Spacing, wording or non-critical visual inconsistency	Track and fix when practical

Defect Workflow

- Report with evidence → Triage → Assign → Fix → Verify → Close/Defer
- The original test case is rerun after the fix.
- Regression tests are added when the defect exposes a reusable failure mode.
- A defect cannot be closed if its linked acceptance criterion still fails.

2.6 Suspension Criteria and Resumption

Suspend development/testing when	Resume when
Walking skeleton, CI/CD baseline or required shared environment is unavailable.	Root cause is fixed and a scoped smoke test passes.
Gateway/chat-core integration is broken in a way that blocks dependent tests.	The integration path is restored and relevant CI checks are green.
A Critical defect affects authentication, authorization, data integrity or safe workflow execution.	The defect is fixed, independently verified and security-sensitive behavior is retested.
Required mock data/services are unavailable or invalid.	Required services/data are restored and smoke checks pass.
Required test configuration or credentials are invalid/exhausted.	Configuration is corrected without exposing secrets; affected tests are rerun.

2.7 Test Completeness

- All slice acceptance criteria pass or have an explicitly recorded, team-approved deferral.
- FR-1 through FR-19 are 100% mapped to test cases and results.
- All four sprint exit criteria pass.
- No open Critical or High defects remain at project closure.
- Main CI is green and deployment gates have been exercised.
- Performance, degradation, recovery and authorization checks meet their defined targets.
- The 20-session demonstration/rehearsal is completed.

2.8 Non-Functional Test Method

Performance and Load

The performance target is p95 response time below 2 seconds, excluding artificial mock-API delay, with 20 concurrent sessions and no noticeable degradation. The test should exercise the three role packs and representative read/write conversation paths.

Measure	Record / acceptance
Concurrency	20 concurrent sessions
Response latency	Record p50, p95 and p99; acceptance target is p95 < 2 seconds excluding artificial mock delay
Errors	Record request failures/timeouts and identify whether failures are application, dependency or environment related
Throughput	Record completed requests/workflows during the run
Workload mix	Distribute sessions across HR, IT and Admissions representative flows
Evidence	Store test date, target build/version, workload, dataset, latency summary, errors and observed degradation

Important boundary: this is a 20-session demonstration-scale performance target, not a claim of production-scale capacity. p50/p99 and throughput are recorded for diagnosis and trend analysis; only the stated p95 and 20-session criteria are acceptance targets.

Security and Authorization

- Verify JWT enforcement on protected routes.
- Verify expired/invalid/missing tokens are rejected appropriately.
- Verify role admins cannot access another role pack's data or configuration.
- Verify input is validated and sanitized before NLU processing or persistence.
- Verify record-affecting actions cannot execute without explicit confirmation.
- Verify sensitive credentials/secrets are never stored in source control or test evidence.
- Verify audit logs capture business-significant actions without exposing secrets.

Reliability and Degradation

- Stop or fault a mock dependency and verify the application returns a controlled response rather than an unhandled failure.
- Make TTS unavailable and verify talk-mode fallback behavior.
- Restart the backend during an active session and verify the defined session-recovery behavior.
- Verify missing records produce the defined 'not found' or next-step behavior.

3. Test Deliverables

#	Deliverable	Description	Owner	Timing
1	Software Test Plan	Scope, strategy, responsibilities, objectives, environments and completion criteria	Team	Baseline; revise on material scope change
2	Slice-level test specifications	Positive, negative and boundary cases derived from acceptance criteria	Slice owner + peer	During slice implementation
3	Automated test suites / CI reports	Backend pytest, frontend Jest/RTL and CI run evidence	All members	Continuous
4	Component/UI evidence	Shared message components and admin UI state verification	Relevant owner + peer	As UI components are built
5	API/contract test collection	Endpoint envelopes, status codes, validation, authentication and rate-limit checks	Gateway/endpoint owners	From gateway availability; regression thereafter
6	Rasa NLU reports	Intent classification, entity extraction where applicable and fallback behavior	HR/IT/Admissions owners	Initial pack setup and after retraining
7	Requirements Traceability Matrix	FR → slice → test case → result → defect/status mapping	Shared team	Living; reviewed every sprint
8	Defect log	Severity, priority, evidence, linked test case, status and resolution	All members	Continuous
9	Sprint test summary	Pass/fail against sprint exit criteria, defects and carry-over risks	Shared team	End of each sprint
10	Performance/load report	20-session results, latency percentiles, errors and observations	Performance test owner + peer	Sprint 4, early enough to fix findings
11	Security/authorization notes	JWT, role scoping, validation, confirmation and credential-handling results	Shared team	During relevant slices and final regression
12	Peer-review evidence	Reviewer identity, reviewed acceptance criteria and review result	Non-implementing team member	Before slice completion
13	UAT/demo checklist	FR-by-FR walkthrough and 20-session demonstration/rehearsal	All members	Final Sprint 4 window
14	Final test closure report	Coverage, final results, defects, risks and completion confirmation	All members	End of Sprint 4

3.1 Test Result Status Values

Not Run — no execution yet; **Pass** — acceptance criterion satisfied with evidence; **Fail** — criterion not satisfied; **Blocked** — execution prevented by an unmet entry condition; **Deferred** — intentionally postponed with a documented team decision.

4. Resource & Environment Needs

4.1 Testing Tools

Category	Testing / method	Purpose
Backend tests	pytest	Unit, integration and API behavior
Frontend tests	Jest + React Testing Library	Component/UI behavior
UI verification	Storybook-style component checks	Shared message components and admin views
API testing	pytest + httpx; curl/Postman-equivalent scripts where useful	Contract, validation, auth and error behavior
Rasa testing	Rasa test/evaluation output	Intent/entity/fallback behavior
CI/CD	GitHub Actions	Automated lint, tests and build gates; deployment verification
Containers	Docker + Docker Compose	Repeatable local integration environment
Database	Alembic + PostgreSQL/Neon	Migration and schema verification
Performance	Custom concurrent-session harness	20-session load/performance target
Version control	Git + pull requests	Peer review, change traceability and controlled merges
Defect/backlog tracking	Jira or project issue tracker	Slice status and defect tracking

4.2 Test Environment Model

VARIO does not use a separate staging tier in the current project model. The deployed demonstration environment is also the System/E2E test target. It must therefore be treated as a **controlled demonstration/test environment**, not as a real customer production environment.

Environment	Purpose	Controls
Local development	Unit and integration testing while implementing a slice	Docker Compose; seeded mock data; no real enterprise credentials
CI	Automated PR gate	Ephemeral runner; lint/test/build; no persistent application state
Deployed demo/test environment	System/E2E, regression, sprint-exit and final rehearsal	Mock enterprise services, seeded demo accounts, controlled test data, smoke test after deployment, rollback/redeploy procedure
Database	Development/demo persistence	Separate test/demo data; migrations verified up/down where applicable

4.3 Deployment/Test Safeguards

- Use seeded demo accounts and mock enterprise services; do not use real employee, applicant or support-ticket data.
- Never place real credentials, tokens or secrets in test cases, screenshots, logs or source control.
- Run a minimal login + chat smoke test after deployment before broad E2E execution.
- If a deployment is unstable, suspend dependent tests rather than recording environment failure as application failure.
- Record the tested build/commit identifier with system-level evidence so results can be reproduced.

4.4 Test Data Management

Data area	Test data rules
Authentication	Use predefined demo identities representing the supported roles; test valid, invalid and expired-token cases without real credentials.
HR	Use seeded leave balances, policies and workflow records sufficient for positive, negative and confirmation tests.
IT	Use seeded ticket records and mocked identity-verification outcomes; include duplicate-request and not-found cases.
Admissions	Use seeded application IDs, program/checklist combinations and calendar data; include unknown IDs.

Data area	Test data rule
Conversation logs	Use synthetic conversations and verify role scoping/pagination.
Performance	Use deterministic seeded data and repeatable workload scripts so runs are comparable.

Appendix A — Functional Requirement Traceability

The traceability model is extended from requirement-to-slice mapping to **Requirement** → **Slice** → **Test Case ID** → **Result** → **Defect**. The table below defines the primary mapping. Detailed execution results are maintained as the project progresses.

FR	Requirement	Primary slice(s)	Primary test ID(s) / tests
FR-1	Classify utterance → intent + confidence	05, 12, 17, 22	TC-CORE / TC-HR / TC-IT / TC-ADM; Integration + E2E
FR-2	Below-threshold confidence → clarification prompt, not workflow	05, 12, 17, 22	TC-CORE; System/E2E
FR-3	New role packs registered via configuration/module setup without core-engine change	10	TC-CORE / TC-E2E; design-time verification
FR-4	Persist conversation context across turns	05	TC-CORE; Integration + System
FR-5	Log every classification and routing decision	05	TC-CORE / TC-ADMIN; Integration
FR-6	Leave balance and leave-application status from HR mock	12–14	TC-HR; System/E2E
FR-7	HR policy FAQs from curated knowledge base	16	TC-HR; System/E2E
FR-8	Guided leave request with confirmation	15	TC-HR / TC-SEC; E2E + safety
FR-9	Return unique reference ID on submission	15	TC-HR; E2E
FR-10	Create IT ticket and return ticket ID	18–19	TC-IT / TC-SEC; E2E
FR-11	Look up existing IT ticket status by ID	18, 20	TC-IT; E2E
FR-12	Guided password reset with mocked identity verification	18, 21	TC-IT / TC-SEC; E2E + safety
FR-13	Prevent duplicate ticket creation from immediate repeated request	19	TC-IT; deduplication E2E
FR-14	Admissions application status lookup	23–24	TC-ADM; E2E
FR-15	Program-based document checklist	25	TC-ADM; E2E
FR-16	Fee/deadline FAQs from configurable calendar	26	TC-ADM; E2E
FR-17	Graceful not-found handling with suggested next step	23–24	TC-ADM / TC-REL; fault injection + E2E
FR-18	Role Admin can manage intents/FAQs only in own role pack	07–08	TC-ADMIN / TC-SEC; API + security
FR-19	Role Admin can view own role-pack conversation/workflow logs	07, 11	TC-ADMIN / TC-SEC; API + security

A.1 Traceability Record Format

Requirement	Slice	Test Case ID	Result	Defect ID	Evidence
FR-x	xx	TC-...	Not Run / Pass / Fail / Blocked / Deferred	DEF-... or —	CI run / screenshot / log / report
NFR-x	xx	TC-...	Not Run / Pass / Fail / Blocked / Deferred	DEF-... or —	Performance/security/reliability evidence

Appendix B — Sprint Exit-Criteria Test Checklist

Sprint 1 — Deployed Walking Skeleton & CD Pipeline (Week 1)

1. A seeded test user can log in through the deployed application and receive a valid VARIO JWT.
2. A pull request triggers the required CI checks; merge to main triggers the defined frontend/backend deployment path.
3. The backend is reachable over HTTPS and the frontend can communicate with it.
4. The PostgreSQL/Neon schema migrates successfully in the intended test environments.
5. Docker Compose can bring up the required local stack for each team member.
6. A visible deployment smoke change reaches the live demonstration/test environment through the normal pipeline.

Sprint 2 — Role Packs & Read Flows (Weeks 2–3)

1. HR, IT and Admissions role packs classify their primary lookup intents and call their adapters/mocks.
2. Login → conversation → chat request → response card works end-to-end.
3. Leave balance, ticket status and application status flows work in the deployed application.
4. Unknown Admissions application IDs produce graceful not-found/next-step behavior.
5. Session state persists across turns and classification/routing events are logged.

Sprint 3 — Write Workflows + Admin + Voice (Weeks 4–5)

1. HR leave request completes through type/start/end/reason → confirmation → reference ID and is audit logged.
2. IT ticket creation works with duplicate suppression.
3. IT password reset uses mocked identity verification and the required safety/confirmation behavior.
4. Admin intent/template/variant management enforces role-pack scoping.
5. Talk mode, browser STT/TTS path and TTS-unavailable fallback are verified.

Sprint 4 — Cross-Cutting Services, Hardening & Demo (Weeks 6–7)

1. Rephraser preserves placeholders and produces reviewable variants/training phrases.
2. Admin retraining can be triggered, monitored and completed with the expected state transition.
3. Role-scoped conversation logs are readable with intent/confidence details.
4. p95 response time is below 2 seconds excluding mock delay for the defined performance test, with 20 concurrent sessions and no noticeable degradation.
5. Graceful degradation is verified for mock/TTS failures and session recovery is verified after a transient backend restart.
6. Every FR has at least one passing automated or documented manual test.
7. The final FR-by-FR demonstration/rehearsal is completed with team sign-off.

Appendix C — Test Risk Register

ID	Risk	Impact	Likelihood	Mitigation / Response	Owner
R-01	Shared deployed environment is unavailable or unstable	High	Medium	Use local/CI tests first; run deployment smoke test; suspend downstream E2E when environment entry criteria fail	All members
R-02	Authentication/authorization defect allows cross-role access	Critical	Low/Med	Dedicated positive/negative JWT and role-scope tests; peer review security-sensitive changes; block closure on failure	Rohith + peer
R-03	Mock service behavior hides integration defects	High	Medium	Use fault injection, not-found cases, malformed inputs and adapter-level tests in addition to happy paths	Role owner + peer
R-04	Performance test is run too late to fix findings	High	Medium	Run performance work early in Sprint 4; preserve time for diagnosis and rerun	Pavan + peer
R-05	Test data becomes inconsistent across runs	Medium	Medium	Use seeded deterministic data and document reset/setup steps	All members
R-06	Secrets appear in logs, screenshots or test configuration	Critical	Low	Use mock/demo credentials only; inspect evidence; never commit secret values	All members
R-07	Regression caused by shared-core change breaks role packs	High	Medium	Run role-pack smoke/regression suite on shared-core changes and before sprint exit	Rohith + role owners
R-08	CI/deployment failure is mistaken for application failure	Medium	Medium	Record environment status and build identifier; separate infrastructure failure from functional defect	Pavan + peer

C.1 Risk Review Rule

Risks are reviewed at each sprint boundary and whenever a Critical/High defect, architecture change, deployment change, or major dependency failure changes the test conditions.

Appendix D — Team Review

The four team members are jointly responsible for approving the test evidence. A member should not be the sole approver of their own implementation. For each sprint and for final closure, the team records the reviewed scope, open defects, deferred items and decision.

Final Test Closure Checklist

- All FR-1 through FR-19 have linked test cases and recorded results.
- All sprint exit criteria are satisfied.
- No open Critical or High defects remain.
- Performance, security, reliability and role-scoping checks are complete.
- The deployed demonstration/test environment has passed final smoke checks.
- All deferred Medium/Low issues have a documented team decision.
- The four team members have reviewed the final evidence and signed the closure record.

End of Document — V.A.R.I.O. Software Test Plan — Team PIPELINE